Programación en Shell. Características Generales de los Shells.



Autor: Francisco de Asís Conde Rodríguez / Lina García Cabrera Copyright:

Índice

Índice **1**
1. Introducción **2**
2. Tipos de Shells **2**
2.1. El Shell Bourne (sh) 2
2.2. El Shell C y Shell Korn 3
2.3. El Shell Bash (bash) 4
2.4. Los shell Zsh (Z Shell) y Fish 5
3. Guiones Shells **6**
3.1. Interpretación de los guiones de Shell 8

1. Introducción

Un `shell` o **intérprete de órdenes** es un **programa que interpreta y ejecuta las órdenes conforme se proporcionan desde el terminal**. No se requiere ningún privilegio especial para ejecutar un shell; para el sistema operativo UNIX un shell es como cualquier otro programa.

Entre las características más comunes de un shell están

- la interpretación de **guiones de shell** (***shell scripts***),
- la **expansión de comodines** en nombres de archivos,
- la combinación de órdenes para formar **interconexiones (pipes)**,
- la **recuperación de órdenes previas**,
- las construcciones **condicionales y los ciclos**, y
- las **variables** para crear abreviaturas.

Un **guion de shell** (***shell script***) es un **archivo (un fichero de texto) que contiene secuencias de órdenes de shell**, igual que si se hubieran tecleado; los guiones de shell permiten adecuar el entorno de trabajo sin tener que realizar programación «real».

2. Tipos de Shells

Existen diversos tipos de shells en los sistemas Unix y Linux, cada uno con su propia historia, sintaxis y conjunto de características. Entre los más conocidos se encuentran el *shell Bourne* (`sh`), el *shell Korn* (`ksh`), el *shell C extendido* (`tcsh`), el *shell C* (`csh`) y el *shell bash* (`bash`).

A ellos se suman, en épocas más recientes, el **Z Shell** (**zsh**) y el **Fish** (Friendly Interactive Shell), que representan la evolución moderna de los intérpretes de comandos. Los tres primeros —Bourne, C y Korn— suelen estudiarse de forma más breve, ya que, aunque son fundamentales en la historia de Unix, muchos de sus conceptos han sido heredados por shells más actuales como Bash o Zsh.

Algunos recursos de shell son **intrínsecos al estilo de trabajo de UNIX**, y además, comunes a todos los shells, ya se han ido presentando en los temas anteriores. Entre ellos destacan las **interconexiones** (pipes) y las **formas básicas de redireccionamiento**, que permiten conectar la salida de unos comandos con la entrada de otros y controlar el flujo de datos en el entorno del terminal.

2.1. El Shell Bourne (sh)

El shell Bourne (sh) formó parte de la séptima edición de UNIX (1979) y es el más antiguo de los shells principales. Fue diseñado por Stephen Bourne en los laboratorios Bell y se convirtió en el estándar para la ejecución de scripts del sistema. Su sintaxis se mantiene

como base del estándar POSIX, por lo que sigue siendo la referencia de compatibilidad para todos los shells posteriores.

Los siguientes recursos son esenciales y están disponibles en el sh original, así como en todos los shells POSIX modernos (incluyendo Bash, Ksh y Zsh):

- **Operadores de Flujo de Control**: Operadores para la ejecución en segundo plano (**`&`**), encadenamiento secuencial (**`;`**), o ejecución condicional de órdenes (**`&&`**, **`||`**).
- **Estructuras de Control de Flujo**: Enunciados (**`if`**, **`case`**) y bucles (**`for`**, **`while`**) para repetir la ejecución de órdenes e iterar a lo largo de secuencias de valores.
- **Variables Sustituibles**:
    - **Variables Nombradas**: Variables definidas por el usuario (ej. `VAR='valor'`).
    - **`Parámetros Posicionales`**: Variables numeradas (ej. **`$1`**, **`$2`**) incializadas automáticamente que contienen los argumentos pasados a una orden o función.
- **Exportación de Variables**: La capacidad de utilizar la orden **`export`** para hacer que variables específicas estén disponibles para los procesos hijo que se ejecutan desde el shell actual.
- **Entrecomillado** (Quoting): Tres formas principales de entrecomillado para controlar la expansión y sustitución de caracteres especiales: comillas simples (**`'...'`**), comillas dobles (**`"..."`**) y comillas invertidas (**`` `...` ``** o **`$()`**).
- **Subshells**: La ejecución de órdenes en entornos separados y aislados (subprocesos) utilizando paréntesis (...).
- **Archivos de Inicialización**: La ejecución de órdenes contenidas en archivos específicos (como **`~/.profile`** o **`~/.bashrc`**) antes de que el shell lea cualquier entrada interactiva o de script.
- Captura de Señales: La orden **`trap`** para capturar señales del sistema (como SIGINT por Ctrl+C o SIGTERM) y ejecutar órdenes específicas cuando ocurren.

2.2. El Shell C y Shell Korn

El **shell C** (`csh`) fue desarrollado por Bill Joy en la Universidad de California (Berkeley), se convirtió en el principal shell de BSD UNIX. Su sintaxis se inspira en el lenguaje C, lo que lo hacía atractivo para programadores. Muchos usuarios consideraban que el shell C era más cómodo para el trabajo interactivo, mientras que el shell Bourne resultaba más adecuado para escribir guiones o scripts. Afortunadamente, la compatibilidad entre shells permite ejecutar scripts de un tipo mientras se utiliza otro en una sesión interactiva con el usuario.

El **shell Tcsh** (`tcsh`) es una versión mejorada de csh, que incorporó edición de línea, autocompletado y un historial de comandos más avanzado.

El **shell Korn** (`ksh`) fue desarrollado por David Korn en los laboratorios Bell a comienzos de los años ochenta, es una extensión del shell Bourne que incorpora muchas de las mejores características del C Shell, además de innovaciones propias como el control de trabajos y las funciones internas. Aunque no se incluye por defecto en todas las distribuciones de

Unix, puede instalarse fácilmente y ofrece un equilibrio muy sólido entre potencia y compatibilidad.

2.3. El Shell Bash (bash)

Con la popularización de Linux y del proyecto GNU, apareció el **shell Bash** (Bourne-Again Shell), desarrollado por Brian Fox en 1989. Bash es una versión mejorada, libre y gratuita del shell Bourne, que incluye muchas de las características del Korn y del C Shell, además de funcionalidades adicionales orientadas tanto al scripting como al uso interactivo. Su nombre es un juego de palabras con "*Bourne-again shell*" (el shell renacido de Bourne).

Gracias a su licencia libre y su compatibilidad con POSIX, Bash se ha convirtido en el shell por defecto en la mayoría de distribuciones Linux y en uno de los intérpretes de comandos más utilizados, incluyendo Ubuntu, y también está disponible en otros sistemas Unix y en Windows (a través de WSL).

Bash está basado en el **Shell Bourne** (`sh`) y puede considerarse un superconjunto compatible: mantiene la sintaxis de scripting POSIX de sh, pero añade numerosas mejoras tanto para el uso interactivo como para la programación avanzada.

Característica de Bash	Utilidad para el Administrador de Sistemas
Historial de Órdenes Avanzado	*Permite buscar comandos anteriores (Ctrl+r) y navegar por la historia, lo que es esencial para la eficiencia y la auditoría rápida.*
Edición de Línea de Comandos	*Facilidades como moverse con el cursor, eliminar palabras (Ctrl+w) o ir al inicio/fin de la línea, lo que reduce drásticamente los errores de escritura.*
Completado Programable (Tab)	Bash no solo autocompleta archivos, sino también órdenes, argumentos, nombres de variables y opciones específicas de comandos, como apt [Tab] install .
Control de Trabajos (Job Control)	La capacidad de conmutar entre procesos y controlar su avance (Ctrl+z para suspender, **fg** para traer al frente, **bg** para enviar al fondo). Fundamental para la multitarea en la terminal.
Alias y Funciones de Shell	Permite crear atajos de forma sencilla (ej. alias ll='ls -alF') sin necesidad de escribir un script completo, mejorando la productividad interactiva. En múltiples distribuciones se

	pueden dejar permanentes en el fichero .bash_aliases
Expansiones y Sustituciones Avanzadas	Formas más flexibles de manipular cadenas y variables (ej. sustitución de parámetros, aritmética simple con $(()), o expresiones regulares básicas).
Operadores y Estructuras de C	Sintaxis adicional para programación que no existía en sh, como los operadores de comparación de estilo C ([[$a -gt 10]] vs. el antiguo [$a -gt 10]).

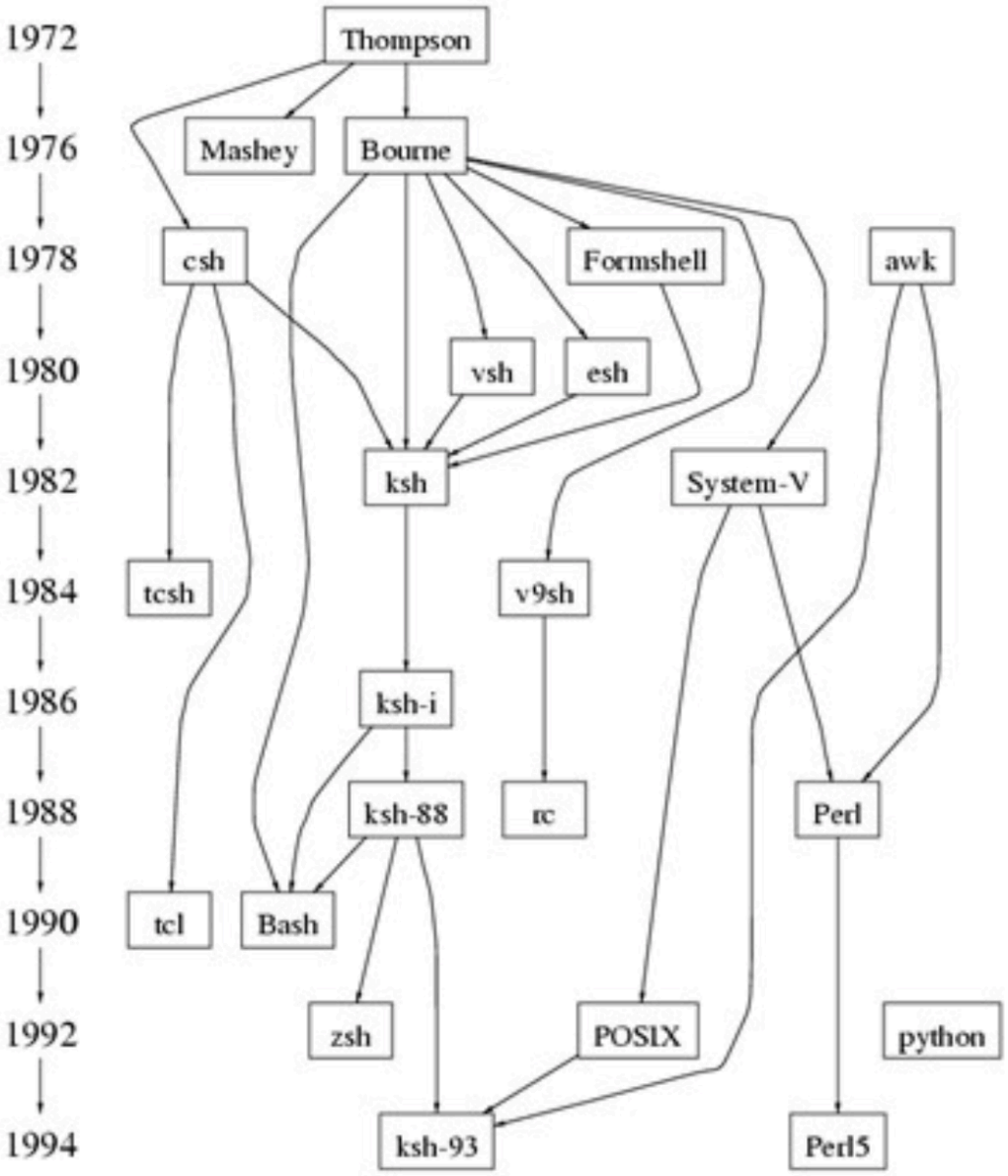
2.4. Los shell Zsh (Z Shell) y Fish

El **Z Shell** o **`zsh`** es un intérprete de comandos desarrollado por Paul Falstad en 1990. Pertenece a la familia de shells compatibles con Bourne (sh) y combina características de Bash, KornShell (ksh) y C Shell (csh), añadiendo potentes mejoras en usabilidad, personalización e interacción.

Zsh mantiene una alta compatibilidad con la sintaxis POSIX, por lo que la mayoría de los scripts escritos para Bash pueden ejecutarse sin modificaciones. No obstante, Zsh destaca sobre todo por su **uso interactivo**, gracias a su avanzado sistema de autocompletado, su gestión de historial y sus amplias opciones de configuración.

Desde macOS Catalina (versión 10.15, lanzada en 2019), Apple adoptó Zsh como shell predeterminado en sustitución de Bash, debido a su licencia más permisiva (MIT) y a su mayor potencia para usuarios interactivos. Esto ha impulsado notablemente su popularidad también fuera del entorno Linux.

Por último, el **Fish** (Friendly Interactive Shell), introducido en 2005, representa una nueva generación de shells enfocados en la **usabilidad y la experiencia del usuari**o. Ofrece autocompletado inteligente, resaltado de sintaxis y sugerencias contextuales sin necesidad de configuraciones adicionales. Sin embargo, no sigue completamente el estándar POSIX, por lo que se utiliza sobre todo de forma interactiva y no tanto para scripting del sistema.



3. Guiones Shells

Los **guiones de shell**, también llamados procedimientos de shell o simplemente ***scripts***, permiten adecuar el entorno añadiendo órdenes propias. Aunque la escritura de un guion de shell requiere un poco de programación, es mucho más fácil que escribir un programa en C o en C++.

Los guiones de shell cortos son fáciles de crear y ejecutar.

Por ejemplo, suponga que desea definir una orden **`dir`** que produzca un listado de directorios con un formato más agradable que el que se obtiene por defecto. Supondremos que usted usa un subdirectorio **`bin`** que cuelga directamente de su directorio base para

almacenar su colección personal de programas y guiones de shell. Puede crear la orden dir si teclea

```
$ cat > $HOME/bin/dir

ls -cF $@

ctrl-d

$ chmod +x $HOME/bin/dir

$ ./bin/dir

$ source ./bin/dir
```

la línea **cat** copia la segunda línea en el archivo **dir**; la línea **ls** define el significado de **dir**; y la línea **chmod** hace que el archivo sea ejecutable (condición necesaria para ejecutar un guion de shell. Los guiones de shell también deben tener permiso de lectura, pero lo más común es que un archivo de nueva creación tenga este permiso por omisión). Este guion de una línea sigue los convencionalismos del shell Bourne.

Después de crear esta orden, puede obtener el listado mejorado si teclea dir seguido por un conjunto de nombres de archivos y directorios. Los nombres pueden incluir comodines. Esta orden también funciona si solamente teclea dir; en este caso obtendrá el contenido del directorio actual. Una versión más elaborada de dir podría, por ejemplo, reconocer diversas opciones, e.g. añadir parámetros adicionales para controlar el comportamiento

Aunque el guión es el mismo, observe qué ocurre al lanzarlo de formas diferentes:

```
$ ~/bin/dir

$ source ~/bin/dir
```

No producen exactamente la misma salida. Esto es porque en la primera ejecución **dir** se ejecuta en un subshell y se pierde la variable de entorno que establece el esquema de colores, esta variable es **LS_COLORS**. En el segundo caso no se crea un subshell y por eso la salida es coloreada.

La orden **source**. Cuando se ejecuta desde una shell un comando o un script, se crea un subproceso (proceso hijo) de la shell que ejecuta el comando o el script (proceso padre). Si el script que ejecuta el proceso hijo crea o modifica alguna variable de entorno, estos cambios o variables desaparecen cuando finaliza el comando o script. Si deseamos que dichos cambios permanezcan, podemos utilizar el comando de Bash **source**. Este comando hace que el proceso o comando se ejecute sin crear ningún proceso hijo, de forma que los cambios efectuados en las variables de entorno y demás, se mantengan al finalizar el archivo.

3.1. Interpretación de los guiones de Shell

Cuando un shell detecta una orden que no es intrínseca, es decir, que **el shell no ejecuta directamente**, llama al sistema operativo para que la ejecute. La orden puede ser un programa compilado o un guion de shell. En el segundo caso, el sistema operativo tiene que seleccionar un shell para ejecutar el guion.

La forma en que el sistema operativo realiza su selección depende del sistema. Por omisión se elige una versión del **shell Bourne** o, en algunas ocasiones, el shell Korn (que es casi completamente compatible con aquel). La compatibilidad con el shell Bourne es esencial, porque históricamente éste era el único disponible en la séptima edición; varios guiones de uso común suponen, sin indicación explícita, que son interpretados por el shell Bourne.

La mayoría de los sistemas se adhieren al convencionalismo de BSD UNIX que especifica que **la primera línea de un guion de shell** tiene la forma

```
#!/ruta/shell
```

donde `shell` es la **ruta absoluta del shell** que se usa para interpretar el guion. El espacio en blanco después de '**#!**' es opcional.

Por ejemplo, si se escribe un guion usando el conjunto de órdenes del **`shell bash`**, éste **deberá comenzar con la línea**[1]**:**

```
#!/bin/bash
```

De hecho, usted puede hacer que **un guion sea interpretado por cualquier programa**.

Por ejemplo, suponga que emite una orden que nombra a un archivo ejecutable, cuya primera línea es

```
#!/usr/sibila/leer.entrañas
```

Entonces, el programa **`leer.entrañas`** del directorio base de `sibila` asume el control, usando como entrada el archivo ejecutable. Por supuesto, **`leer.entrañas`** debe tener la inteligencia suficiente para reconocer la primera línea como comentario e ignorarla.

Algunos sistemas que no respetan el convencionalismo '**#!**' ofrecen otra manera de etiquetar los guiones de shell:

- si el primer carácter es '**:**', se considera como un guion del shell Bourne;
- si el primer carácter es '**#**', se considera como un guion del shell C, csh.

[1] **#!/bin/bash Shebang** es, en la jerga de Unix, el nombre que recibe el par de caracteres **#!** que se encuentran al inicio de los programas ejecutables interpretados. En algunas ocasiones se le denomina también *hash-bang* o *sharpbang*. A continuación de estos caracteres se indica la ruta completa al intérprete de las órdenes contenidas en el mismo (el SheBang más habitual suele ser "**`#!/bin/bash`**", que es una llamada al intérprete de comandos Bash). Este método estándar permite que el usuario pueda ejecutar un programa interpretado como si se ejecutara un programa binario.

Todos los shells comunes proporcionan una opción **`-v`** para presentar cada línea de entrada conforme se lee y una opción **`-x`** para presentar las órdenes cuando se ejecutan. Como veremos, estas opciones son muy útiles para depurar guiones.

En las prácticas se programará en el shell bash.